Mini-project 2: measurement

Start Assignment

- Due Friday by 5pm
- Points 10
- Submitting a text entry box or a file upload
- File Types pdf

Overview

The goal of this assignment is to get some experience doing benchmarking by measuring the performance of various operating system and interprocess communication mechanisms. You will design various experiments, build simple tools, and carry out a methodical experiment, summarize the results, and draw conclusions.

Be careful! Benchmarking is a subtle and tricky business; things that look simple on first glance will often turn out to be quite intricate.

Group work

This project will be done in groups of 2.

Description

The most basic of IPC mechanisms on UNIX is the pipe; it has been around since the earliest versions of UNIX. The `pipe` system call is executed by a process to create both ends of a uni-directional communication channel. This channel is a stream of bytes that ensures ordering and correct delivery. The pipe, when combined with a `fork` operation, allows two processes to pass messages.

Your job is to measure the latency and bandwidth of pipes in an operating system - could be Linux, MacOS, Windows, IOS, Android, or anything else.

The Measurements

1. Clock precision

The accuracy and granularity of the timer you use will often have a large effect on your measurements. Therefore, you should use the best timer available. There are several APIs you can try: `gettimeofday` and `clock_gettime`.

In addition, on x86 platforms a highly accurate **cycle counter** is available. The instruction to use it is known as **rdtsc**, (**Wikipedia article (http://en.wikipedia.org/wiki/RDTSC)**) and it returns a 64-bit cycle count. By knowing the cycle time, one can easily convert the result of rdtsc into a useful time. **Here (http://queue.acm.org/detail.cfm?id=3036398)** is an article with information about fine-grained timing measurements.

A few caveats:

1. If the processor can automatically vary the clock speed, the timestamp counter may not reflect real time.
2. On a multicore system, different processor cores may have different values for the timestamp; you can only compare values on a single core and not across cores.

Hence, the first thing you should do is: figure out how to use `rdtsc` or its analogue (you can use google to find out more about it). Once you know how to call it and get a cycle count, convert the result to seconds and measure how long something takes (e.g., a program that calls sleep(10) and exits should run for about 10 seconds. Confirm your results make sense by comparing it to a less accurate but reliable counter such as **gettimeofday. Note that confirmation of timer accuracy is hugely important!** If you don't trust your timer, how can you trust the results of your measurements?

One way to do this is to read the clock value at the start and end of a simple loop. Start with a single loop iteration, then increase the iteration count of the loop until the difference between the before and after samples is greater than zero. Try to get the smallest non-zero positive difference. If a single iteration of a loop takes too much time, try putting simple statements between the two timer calls.

You should try at least 2 mechanisms for measuring time, and determine the resolution (smallest time value) you can accurately measure.

More information on time **here (http://queue.acm.org/detail.cfm?id=2878574)** .

Inter-Process Communication Time:

You will measure the following characteristics:

Message latency: Latency is the time for some activity to complete, from beginning to end. For message passing, it is the time from the start of a send to the completion of a receive. Since the clocks on two different cores (if using `rdtsc`) may not be sufficiently aligned, the easiest way to measure message latency is to measure the time it takes to complete a round-trip communication (and divide by two).

Measure latency for a variety of message sizes: 4, 16, 64, 256, 1K, 4K, 16K, 64K, 256K, and 512K bytes.

Throughput: Throughput is the amount data that is sent per unit time. In this case, a round trip

measure is not necessary; you can sent a return message when the entire transfer amount has been sent. Send a **large-enough total quantity** of data such that the single "ack" response contributes a small amount of time compared to the whole transfer.

Measure throughput for a variety of message sizes, the same as above for latency.

Where the time goes

The final part of this project is to determine *why* each pipes performs as they do -- where does the time go? Here, you must be a detective and use many available tools to find out where the time goes. This is your opportunity to research what is available and learn how to use them. Some suggestions

- Processor performance counters -- **Perf (https://perf.wiki.kernel.org/index.php/Main_Page)**
- System call tracing - **strace (http://man7.org/linux/man-pages/man1/strace.1.html)**
- Kernel tracing --- **Ftrace (http://elinux.org/Ftrace)**

As much as possible, you should break down what the causes of performance are: time spent in your test program, in various parts of the kernel, or in expensive processor operations such as context switches or communicating memory across processor cores. Ideally, you can produce a chart or table showing the contribution of these components to performance

The Experimental Method

Computer Scientists are notably sloppy experimentalists (this applies to both implementers and measurers). While we do a lot of experimental work, we typically do not follow good experimental practice. The experimental method is a well-established regimen, used in all areas of science. The use of the experimental method keeps us honest and gives form to the work that we do.

The basic parts of an experiment are:

1. Identify your variables:
   Variables are things that you can observe and quantify. You need to identify which variables might be related and whether a variable is a cause (i.e., the message size of a send operation) or the effect (e.g., the time to complete the send). Even though this sounds obvious, you should consciously identify the variables in each experiment that you perform.
2. Hypothesis:
   The hypothesis is a guess (we hope, an educated guess) about the outcome of the experiment. The hypothesis needs to be worded in a way that can be tested in an experiment, so it should be stated in terms of the experimental variables.
3. Experimental apparatus:
   You need to obtain the necessary equipment for your experiment. In this case, it will be the needed computer and software.
4. Performance of experiment and record the results:

This part is the one that we typically think of as the real work. Note that several important steps come before it.

5. Summarize the results:
   Summarization means putting the data in a form that you can understand. You might put the data in tables, graphs, or use statistical techniques to understand the raw data. If you are using averages, make sure to read **Jim Smith's paper (http://www.cs.wisc.edu/~bart/736/papers/p1202-smith.pdf)** in the October 1988 issue of CACM, **Jose Albertal's Lecture (http://webdocs.cs.ualberta.ca/~amaral/Amaral-LCTES2012.pdf)**, or **Gernot Heiser's web page (http://www.cse.unsw.edu.au/~gernot/benchmarking-crimes.html)** (there are many types of means, and you need to use the right one)! However, as a warning, you probably do **not** want to use averages; taking the minimum makes much more sense in this case.
6. Draw conclusions:
   Note that performing the experiment and summarizing the results are separate steps and both come *before* you draw conclusions. To present honest and understandable results, we must present the basic data first (so that the reader can draw their own conclusions) before we insert our bias.

   The experimental method has more subtleties than this (such as trying to account for experimenter and subject biases), but the above description is sufficient for basic computer measurement experiments.

Writeup

Each team of two will submit a joint writeup. The writeup should document design and measurement choices. Use this opportunity to evaluate your measurement methodology and communication mechanism design.

Please write a 2-page paper, 2 column, 11-point font, 1-inch margins (conference style!).

The paper must contain the following parts:

- Title:
  The title should be descriptive and fit in one line across the page. Interesting titles are acceptable, but avoid overly cute ones.
- Body:
  This is the main part of the paper. It should include an introduction that prepares the reader for the remainder of the paper. Assume that the reader is knowledgeable about operating systems. The introduction should motivate the rest of the discussion and outline the approach. The main part of the paper should be split into reasonable sections that follow the basics of the experimental method. This is a discussion of what the reader should have learned from the paper. You can repeat things stated earlier in the paper, but only to the extent that they contribute to the final discussion.

- Figures:
  A paper without figures, graphs, or diagrams is boring. This paper will certainly need several performance tables and graphs. Your paper **must** have figures. Your figures should be easy to understand (no microscopic fonts, labeled axes, etc.)

Do not re-describe the assignment; address the issues described above. The paper must be written using correct English grammar. There should be no spelling mistakes.

Note that your paper will be evaluated on both the technical content and the presentation (as would any paper submitted to a journal or conference).

Here is a list of **writing suggestions (http://www.cs.wisc.edu/~bart/WritingSuggestions.html)** available to help you avoid common mistakes. It is essential that you take a look at these suggestions as you prepare your paper.

A **grading rubric (https://canvas.wisc.edu/courses/516041/files/54238196?wrap=1)** ↓ **(https://canvas.wisc.edu/courses/516041/files/54238196/download?download_frd=1)** is available for the paper.

How to turn it in

Please turn in the paper (pdf format) on Canvas.